

Worksheet No. - 04

Student Name: Sukhleen Kaur

UID: 25MCA20031

Branch: MCA

Section/Group: 25MCA-6B

Semester: 2nd

Date of Performance: 02.03.2026

Subject Name: Advanced Internet Programming Lab **Subject Code:** 25CAP-656

Aim/Overview of the practical: (A). Convert and deploy the employee data application in Enterprise Java Beans and apply the Bean Validation.

Objective: To convert the existing Employee Data Application into an Enterprise Java Beans (EJB) based application in order to implement business logic using EJB architecture, and to apply Bean Validation for ensuring data accuracy, integrity, and consistency before storing employee details in the database.

Procedure/Algorithm/Code

Code

Employee1.java

```
package com.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.JoinColumn;

@Entity

public class Employee1 {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
private int id;

private String name;

private String email;

private double salary;

@ManyToOne

@JoinColumn(name = "address_id")

private Address address;

// Getters and Setters

public int getId() { return id; }

public void setId(int id) { this.id = id; }

public String getName() { return name; }

public void setName(String name) { this.name = name; }

public String getEmail() { return email; }

public void setEmail(String email) { this.email = email; }

public double getSalary() { return salary; }

public void setSalary(double salary) { this.salary = salary; }

public Address getAddress() { return address; }

public void setAddress(Address address) { this.address = address; }

}
```

Address.java

```
package com.model;

import jakarta.persistence.Entity;

import jakarta.persistence.GeneratedValue;

import jakarta.persistence.GenerationType;

import jakarta.persistence.Id;
```

```
import jakarta.persistence.OneToOne;
```

```
import java.util.List;
```

```
@Entity
```

```
public class Address {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private int id;
```

```
    private String city;
```

```
    private String state;
```

```
    @OneToMany(mappedBy = "address")
```

```
    private List<Employee1> employees;
```

```
    // Getters and Setters
```

```
    public int getId() { return id; }
```

```
    public void setId(int id) { this.id = id; }
```

```
    public String getCity() { return city; }
```

```
    public void setCity(String city) { this.city = city; }
```

```
    public String getState() { return state; }
```

```
    public void setState(String state) { this.state = state; }
```

```
    public List<Employee1> getEmployees() { return employees; }
```

```
    public void setEmployees(List<Employee1> employees) { this.employees = employees; }
```

```
}
```

```
}  
  
persistence.xml
```

```
<persistence xmlns="https://jakarta.ee/xml/ns/persistence" version="3.1">
```

```
<persistence-unit name="EmployeePU" transaction-type="RESOURCE_LOCAL">

    <class>com.model.Employee</class>

    <class>com.model.Address</class>

    <properties>

        <property name="jakarta.persistence.jdbc.url"
value="jdbc:mysql://localhost:3306/employee_dbds"/>

        <property name="jakarta.persistence.jdbc.user" value="root"/>

        <property name="jakarta.persistence.jdbc.password" value="abc@123"/>

        <property name="jakarta.persistence.jdbc.driver"
value="com.mysql.cj.jdbc.Driver"/>

        <property name="hibernate.dialect"
value="org.hibernate.dialect.MySQLDialect"/>

        <property name="hibernate.hbm2ddl.auto" value="update"/>

    </properties>

</persistence-unit>

</persistence>
```

EmployeeBean.java

```
package com.bean;

import com.model.Employee1;

import com.model.Address;

import jakarta.persistence.EntityManager;

import jakarta.persistence.EntityManagerFactory;

import jakarta.persistence.Persistence;

public class EmployeeBean {

    private EntityManager em;

    public EmployeeBean() {
```

```
EntityManagerFactory emf =
Persistence.createEntityManagerFactory("EmployeePU");

    em = emf.createEntityManager();

}

// Persist Address

public void addAddress(Address address) {

    em.getTransaction().begin();

    em.persist(address);

    em.getTransaction().commit();

}

// Persist Employee

public void addEmployee(Employee1 employee) {

    em.getTransaction().begin();

    em.persist(employee);

    em.getTransaction().commit();

}

public EntityManager getEntityManager() {

    return em;

}

}
```

Aim/Overview of the practical: (B). Apply one-to-one Entity relationship mapping.

Objective: To implement One-to-One Entity Relationship Mapping in the application using JPA/Hibernate in order to establish a direct relationship between two entities where one record in an entity is associated with exactly one record in another entity.

Procedure/Algorithm/Code

Code

EmployeeServlet.java

```
package com.controller;

import com.bean.EmployeeBean;
import com.model.Employee1;
import com.model.Address;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.persistence.TypedQuery;
import java.io.IOException;
import java.util.List;

@WebServlet("/addEmployee")

public class EmployeeServlet extends HttpServlet {

    private EmployeeBean employeeBean = new EmployeeBean();

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        // Create Address

        Address addr = new Address();

        addr.setCity(request.getParameter("city"));

        addr.setState(request.getParameter("state"));

        // Save Address first

        employeeBean.addAddress(addr);
```

```
// Create Employee
```

```
Employee1 emp = new Employee1();
```

```
emp.setName(request.getParameter("name"));
```

```
emp.setEmail(request.getParameter("email"));
```

```
try {
```

```
    emp.setSalary(Double.parseDouble(request.getParameter("salary")));
```

```
} catch (NumberFormatException e) {
```

```
    emp.setSalary(0);
```

```
}
```

```
emp.setAddress(addr); // link to saved address
```

```
employeeBean.addEmployee(emp);
```

```
// Now display all employees in HTML table
```

```
response.setContentType("text/html");
```

```
StringBuilder html = new StringBuilder();
```

```
html.append("<html><head><title>Employee List</title></head><body>");
```

```
html.append("<h2>Employee Details</h2>");
```

```
html.append("<table border='1'>");
```

```
html.append("<tr><th>ID</th><th>Name</th><th>Email</th><th>Salary</th><th>City</th><th>State</th></tr>");
```

```
// Fetch all employees from DB
```

```
TypedQuery<Employee1> query =
```

```
employeeBean.getEntityManager().createQuery("SELECT e FROM Employee1 e",  
Employee1.class);
```

```
List<Employee1> list = query.getResultList();
```

```
for (Employee1 e : list) {
```

```
        html.append("<tr>");

        html.append("<td>").append(e.getId()).append("</td>");

        html.append("<td>").append(e.getName()).append("</td>");

        html.append("<td>").append(e.getEmail()).append("</td>");

        html.append("<td>").append(e.getSalary()).append("</td>");

        html.append("<td>").append(e.getAddress().getCity()).append("</td>");

        html.append("<td>").append(e.getAddress().getState()).append("</td>");

        html.append("</tr>");

    }

    html.append("</table></body></html>");

    response.getWriter().println(html.toString());

}

}
```

index.jsp

```
<html>

<body>

<h2>Employee Form</h2>

<form action="addEmployee" method="post">

Name: <input type="text" name="name"><br><br>

Email: <input type="text" name="email"><br><br>

Salary: <input type="text" name="salary"><br><br>

City: <input type="text" name="city"><br><br>

State: <input type="text" name="state"><br><br>

<input type="submit" value="Save">

</form>
```


</body>

</html>

SQL Code:

```
CREATE DATABASE employee_dbsd;
```

```
USE employee_dbsd;
```

```
DROP Table employee1;
```

```
DROP Table address;
```

```
CREATE TABLE address (
```

```
    id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    city VARCHAR(50),
```

```
    state VARCHAR(50)
```

```
);
```

```
-- Employee table
```

```
CREATE TABLE employee1 (
```

```
    id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    name VARCHAR(50),
```

```
    email VARCHAR(50),
```

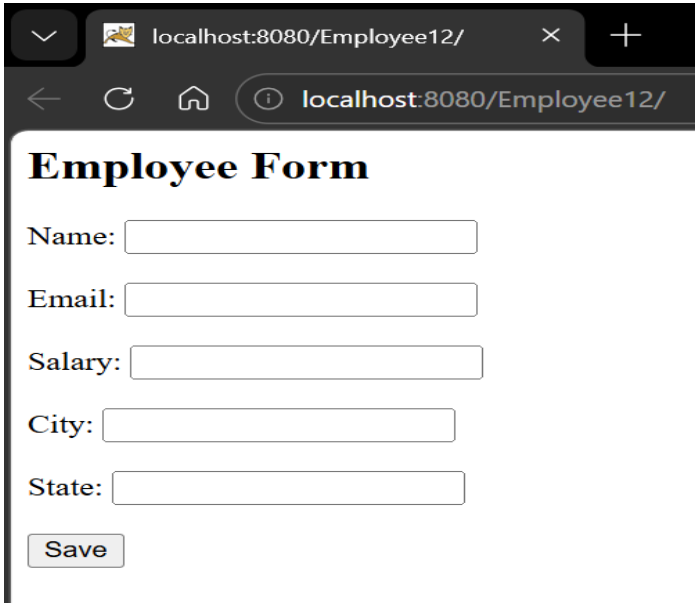
```
    salary DOUBLE,
```

```
    address_id INT,
```

```
    CONSTRAINT fk_address FOREIGN KEY (address_id) REFERENCES address(id)
```

```
);
```

Output



A screenshot of a web browser window showing a form titled "Employee Form". The browser's address bar displays "localhost:8080/Employee12/". The form contains five input fields: "Name:", "Email:", "Salary:", "City:", and "State:". All fields are empty. Below the fields is a "Save" button.

Employee Form

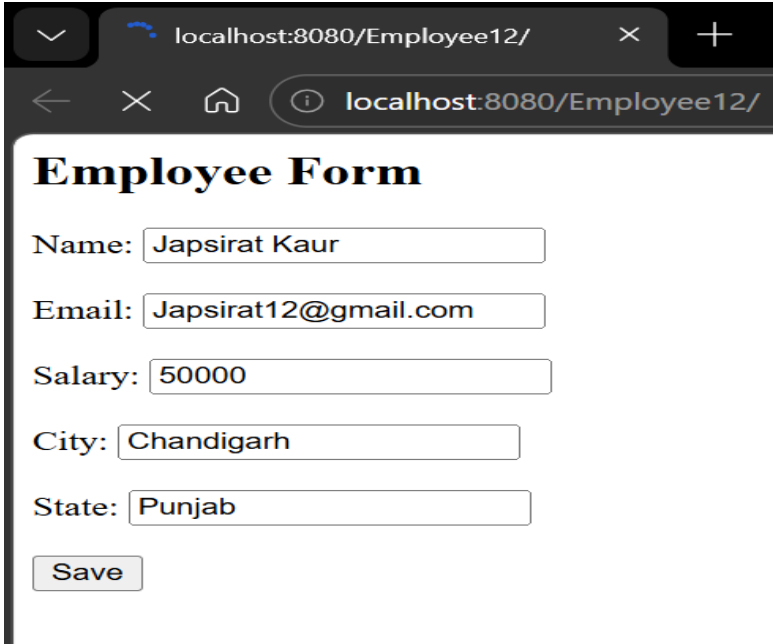
Name:

Email:

Salary:

City:

State:



A screenshot of a web browser window showing the same "Employee Form" as above, but with pre-filled data. The browser's address bar displays "localhost:8080/Employee12/". The form fields are filled with: "Name: Japsirat Kaur", "Email: Japsirat12@gmail.com", "Salary: 50000", "City: Chandigarh", and "State: Punjab". The "Save" button is still present at the bottom.

Employee Form

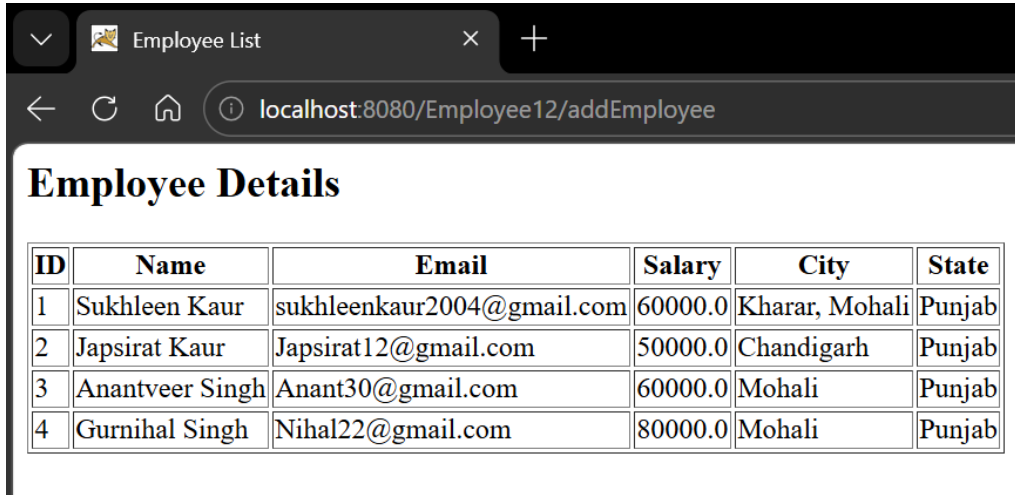
Name:

Email:

Salary:

City:

State:



The screenshot shows a web browser window with a single tab titled "Employee List". The address bar displays "localhost:8080/Employee12/addEmployee". The main content area is titled "Employee Details" and contains a table with 6 columns: ID, Name, Email, Salary, City, and State. The table lists 4 employees.

ID	Name	Email	Salary	City	State
1	Sukhleen Kaur	sukhleenkaur2004@gmail.com	60000.0	Kharar, Mohali	Punjab
2	Japsirat Kaur	Japsirat12@gmail.com	50000.0	Chandigarh	Punjab
3	Anantveer Singh	Anant30@gmail.com	60000.0	Mohali	Punjab
4	Gurnihal Singh	Nihal22@gmail.com	80000.0	Mohali	Punjab

Learning outcomes (What I have learnt):

1. Understood and implement EJB for business logic development.
2. Applied Bean Validation to ensure accurate and consistent data.
3. Implement One-to-One entity relationship mapping using JPA annotations.